



LIMKOKWING LIBRARY API

BASIC API STRUCTURE WITH OPEN-SOFTWARE



MAIMUNATU A. R. MASSAQUOI
OOP ASSIGNMENT

Table of Contents

PART 1 – API DESIGN (LIMKOKWING LIBRARY)	2
Proposed Endpoints, Purpose and Examples	2
Hand drawn sketch	2
PART B – IMPLEMENTATION	4
Screenshots	4
Asynchronous behaviour:.....	5
PART C – REFLECTION	6
How APIs improve efficiency	6
Sustainable Development Goal (SDG Alignment)	6
Challenges.....	6
Future Improvement	6

PART 1 – API DESIGN (LIMKOKWING LIBRARY)

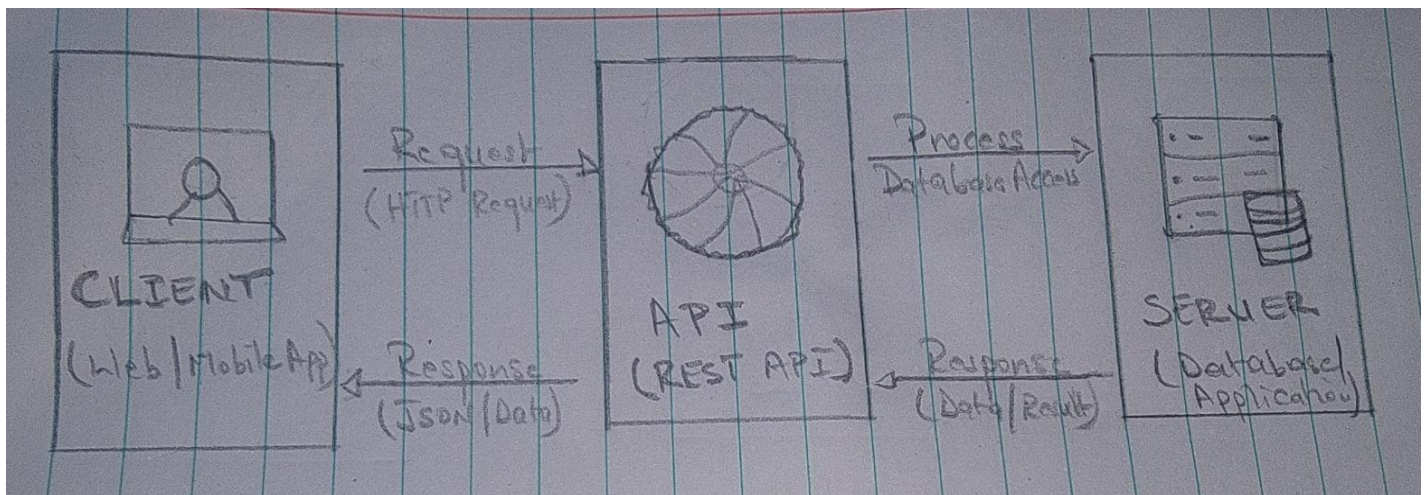
Proposed Endpoints, Purpose and Examples

METHOD	ENDPOINT	PURPOSE
GET	/books/search	Search for books by title, author, or category
POST	/books/borrow	Borrow a book by providing user_id and book_id
POST	/books/return	Return a borrowed book and calculate fines if overdue
GET	/users/overdue	Track overdue books and fines per user

EXAMPLE INPUT	EXAMPLE OUTPUT
/books/search?title=Python	{ "books": [{ "id": 1, "title": "Writing Codes", "author": "Ambrose John", "available": true }] }
{ "user_id": 101, "book_id": 1 }	{ "message": "Book borrowed successfully", "due_date": "2026-05-12" }
{ "user_id": 101, "book_id": 1 }	{ "message": "Book returned", "fine": 5.00 }
/users/overdue?user_id=101	{ "user_id": 101, "overdue_books": [{ "book": "Python Basics", "days_overdue": 2, "fine": 4.00 }] }

Hand drawn sketch

A diagram (hand-drawn) showing how the client, API, and server interact.



Client (Request): A user interacts with a client application (like a mobile app or web browser). The client initiates the process by sending a request (e.g., a GET or POST request) to the API.

API (Messenger): The API acts as an interface layer. it takes the client's request, manages security measures like authentication, and passes the validated instructions to the server.

Server (Processing): The server receives the instructions, processes the logic, and interacts with a database if necessary to retrieve or update data.

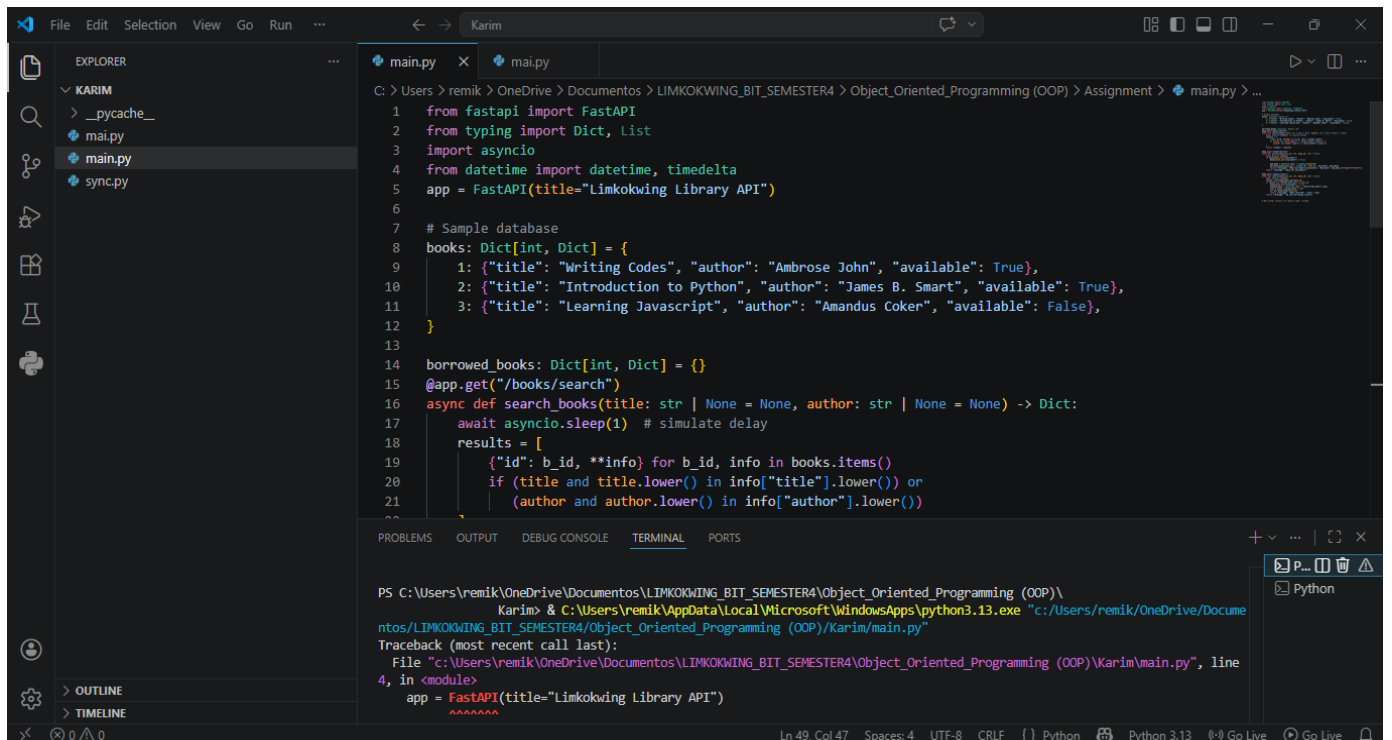
API (Response Delivery): The server sends the result back to the API. The API then formats this data (often as JSON) and delivers it back to the client.

Client (Display): The client application receives the data and updates the user interface

PART B – IMPLEMENTATION

Screenshots

Below are screenshots of a simplified FastAPI script with asynchronous endpoints demonstrating concurrent borrowing and returning.



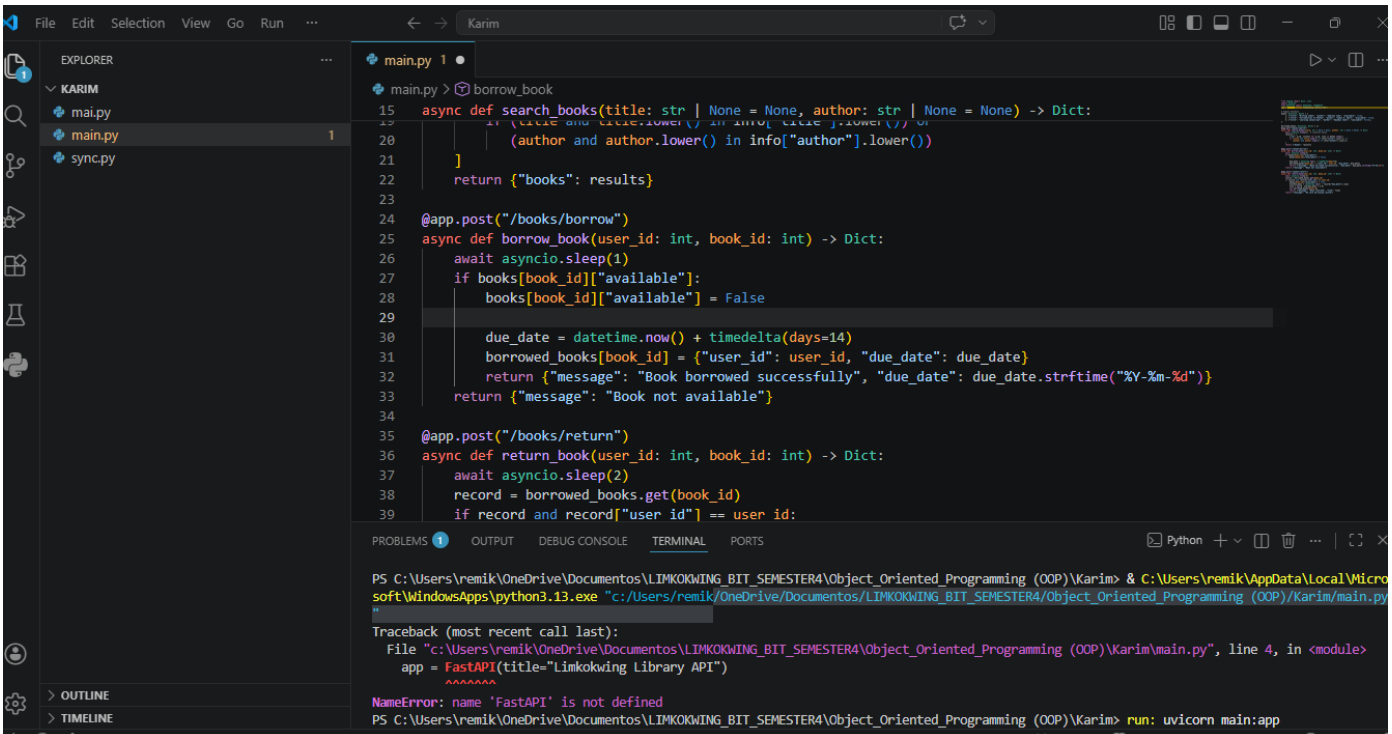
```
1 from fastapi import FastAPI
2 from typing import Dict, List
3 import asyncio
4 from datetime import datetime, timedelta
5 app = FastAPI(title="Limkokwing Library API")
6
7 # Sample database
8 books: Dict[int, Dict] = {
9     1: {"title": "Writing Codes", "author": "Ambrose John", "available": True},
10    2: {"title": "Introduction to Python", "author": "James B. Smart", "available": True},
11    3: {"title": "Learning Javascript", "author": "Amandus Coker", "available": False},
12 }
13
14 borrowed_books: Dict[int, Dict] = {}
15 @app.get("/books/search")
16 async def search_books(title: str | None = None, author: str | None = None) -> Dict:
17     await asyncio.sleep(1) # simulate delay
18     results = [
19         {"id": b_id, **info} for b_id, info in books.items()
20         if (title and title.lower() in info["title"].lower()) or
21            (author and author.lower() in info["author"].lower())
22     ]
23     return {"books": results}
24
25 @app.post("/books/borrow")
26 async def borrow_book(user_id: int, book_id: int) -> Dict:
27     await asyncio.sleep(1)
28     if books[book_id]["available"]:
29         books[book_id]["available"] = False
30
31         due_date = datetime.now() + timedelta(days=14)
32         borrowed_books[book_id] = {"user_id": user_id, "due_date": due_date}
33         return {"message": "Book borrowed successfully", "due_date": due_date.strftime("%Y-%m-%d")}
34     else:
35         return {"message": "Book not available"}
```

PS C:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim> C:\Users\remik\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim/main.py"

Traceback (most recent call last):

File "c:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim/main.py", line 4, in <module>

app = FastAPI(title="Limkokwing Library API")



```
15 async def search_books(title: str | None = None, author: str | None = None) -> Dict:
16     if (title and title.lower() in info["title"].lower()) or
17        (author and author.lower() in info["author"].lower())
18     ]
19     return {"books": results}
20
21 @app.post("/books/borrow")
22 async def borrow_book(user_id: int, book_id: int) -> Dict:
23     await asyncio.sleep(1)
24     if books[book_id]["available"]:
25         books[book_id]["available"] = False
26
27         due_date = datetime.now() + timedelta(days=14)
28         borrowed_books[book_id] = {"user_id": user_id, "due_date": due_date}
29         return {"message": "Book borrowed successfully", "due_date": due_date.strftime("%Y-%m-%d")}
30     else:
31         return {"message": "Book not available"}
```

```
35 @app.post("/books/return")
36 async def return_book(user_id: int, book_id: int) -> Dict:
37     await asyncio.sleep(2)
38     record = borrowed_books.get(book_id)
39     if record and record["user id"] == user_id:
```

PS C:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim> C:\Users\remik\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim/main.py"

Traceback (most recent call last):

File "c:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim/main.py", line 4, in <module>

app = FastAPI(title="Limkokwing Library API")

NameError: name 'FastAPI' is not defined

PS C:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim> run: uvicorn main:app

The screenshot shows a Visual Studio Code editor window with a Python FastAPI application. The Explorer sidebar on the left shows a project named 'KARIM' with files: `__pycache__`, `mai.py`, `main.py`, and `sync.py`. The main editor displays `main.py` with the following code:

```
26 async def borrow_book(user_id: int, book_id: int) -> Dict:
27     due_date = datetime.now() + timedelta(days=14)
32     borrowed_books[book_id] = {"user_id": user_id, "due_date": due_date}
33     return {"message": "Book borrowed successfully", "due_date": due_date.strftime("%Y-%m-%d")}
34     return {"message": "Book not available"}
35
36 @app.post("/books/return")
37 async def return_book(user_id: int, book_id: int) -> Dict:
38     await asyncio.sleep(2)
39     record = borrowed_books.get(book_id)
40     if record and record["user_id"] == user_id:
41         books[book_id]["available"] = True
42         overdue_days = (datetime.now() - record["due_date"]).days
43         fine = max(0, overdue_days * 2.0)
44         del borrowed_books[book_id]
45         return {"message": "Book returned", "fine": fine}
46     return {"message": "No such borrowing record"}
47
48
49 # Run using: uvicorn.run(main()):app --reload
50
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\remik\OneDrive\Documents\LIMKOKWING_BIT_SEMESTER4\Object_Oriented_Programming (OOP)\Karim> pip install fastapi
uvicorn
Defaulting to user installation because normal site-packages is not writeable

Requirement already satisfied: fastapi in C:\Users\remik\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2
kfra8p0\LocalCache\local-packages\Python313\site-packages (0.136.1)
Collecting uvicorn
  Downloading uvicorn-0.46.0-py3-none-any.whl.metadata (6.7 kB)
```

The status bar at the bottom indicates the file is at line 49, column 47, with 4 spaces, UTF-8 encoding, CRLF line endings, and Python 3.13 interpreter.

Asynchronous behaviour:

The `await asyncio.sleep(2)` simulates multiple users accessing or borrowing books at once without blocking others.

PART C – REFLECTION

How APIs improve efficiency

APIs enable Limkokwing Library to automate common activities such as book searches, borrowing, and returns. Staff can access real-time data through a simple web interface without manually updating paper logs. It reduces human error, saves time, and supports quick decision-making. APIs also allow future integration with mobile apps or student portals, improving accessibility and service quality.

Sustainable Development Goal (SDG Alignment)

This project aligns with SDG 4: Quality Education. By digitizing library services, the system supports easier access to educational resources, encouraging lifelong learning and technological literacy among students.

Challenges

The main challenge was handling concurrent requests using asynchronous functions. Structuring JSON responses consistently and simulating database updates in memory also required careful testing. Managing data integrity when multiple users borrow or return the same book simultaneously was particularly tricky.

Future Improvement

If the library expands, I will integrate a real database (e.g., PostgreSQL) and implement JSON web authentication for secure user access, improving scalability and data protection.